```
"""
Daily sector "flow proxy" poster for Discord (webhook version).

Ranks the 11 SPDR sector ETFs by daily % change and shows dollar volume
relative to its 20-day average. This is a PROXY for sector inflows/outflows,
not true fund-flow data.

Setup:
    pip install yfinance requests
    export DISCORD_WEBHOOK_URL="https://discord.com/api/webhooks/..."
    python sector_flows.py --force      # test post right now
"""

import datetime as dt
import os
import sys
from zoneinfo import ZoneInfo

import requests
import yfinance as yf

WEBHOOK_URL = os.environ.get("DISCORD_WEBHOOK_URL", "")
ET = ZoneInfo("America/New_York")
TOP_N = 3

SECTORS = {
    "XLK": "Technology",
    "XLF": "Financials",
    "XLE": "Energy",
    "XLV": "Health Care",
    "XLY": "Consumer Discretionary",
    "XLP": "Consumer Staples",
    "XLI": "Industrials",
    "XLB": "Materials",
    "XLU": "Utilities",
    "XLRE": "Real Estate",
    "XLC": "Communication Services",
}


def fetch_rows():
    data = yf.download(
        list(SECTORS),
        period="3mo",
```

```
        interval="1d",
        auto_adjust=True,
        progress=False,
    )
    close = data["Close"].dropna(how="all")
    volume = data["Volume"].reindex(close.index)
    dollar_vol = close * volume

    last_date = close.index[-1].date()
    rows = []
    for t, name in SECTORS.items():
        try:
            pct = (close[t].iloc[-1] / close[t].iloc[-2] - 1) * 100
            avg20 = dollar_vol[t].iloc[-21:-1].mean()
            rel_vol = dollar_vol[t].iloc[-1] / avg20 if avg20 else float("nan")
            rows.append({"ticker": t, "name": name, "pct": pct, "rel_vo
        except Exception as e:  # skip a ticker if its data is missing
            print(f"Skipping {t}: {e}")
    return last_date, rows


def fmt(r, up):
    dot = "🟢" if up else "🔴"
    return (
        f"{dot} **{r['ticker']}** {r['name']}\n"
        f"  {r['pct']:+.2f}% · {r['rel_vol']:.1f}x avg volume"
    )


def post(last_date, rows):
    rows = sorted(rows, key=lambda r: r["pct"], reverse=True)
    inflows = rows[:TOP_N]
    outflows = rows[-TOP_N:][::-1]  # weakest first

    embed = {
        "title": f"Sector Flows (Proxy) · {last_date:%b %d, %Y}",
        "color": 0x2B6CB0,
        "fields": [
            {
                "name": "📈 Strongest (inflow proxy)",
                "value": "\n".join(fmt(r, True) for r in inflows),
                "inline": False,
            },
            {
                "name": "📉 Weakest (outflow proxy)",
                "value": "\n".join(fmt(r, False) for r in outflows),
                "inline": False,
```

```
            },
        ],
        "footer": {
            "text": "Based on SPDR sector ETF price change and dollar volume vs 2
            "Not true fund flows. Not financial advice."
        },
    }
    resp = requests.post(WEBHOOK_URL, json={"embeds": [embed]}, timeout=15)
    resp.raise_for_status()
    print("Posted to Discord.")


def main():
    if not WEBHOOK_URL:
        sys.exit("Set the DISCORD_WEBHOOK_URL environment variable.")

    force = "--force" in sys.argv
    now_et = dt.datetime.now(ET)

    # Scheduled runs only post during the 4 PM ET hour on weekdays.
    # (Lets us schedule two UTC crons to cover daylight saving changes.)
    if not force and (now_et.weekday() >= 5 or now_et.hour != 16):
        print(f"Not posting window ({now_et:%a %H:%M} ET). Use --force to overrid
        return

    last_date, rows = fetch_rows()

    # Skip market holidays: latest bar must be today's.
    if not force and last_date != now_et.date():
        print(f"Latest data is {last_date}, not today. Market likely closed.")
        return

    if len(rows) < 2 * TOP_N:
        sys.exit("Not enough data returned. Try again later.")

    post(last_date, rows)


if __name__ == "__main__":
    main()
```